SQL Advanced Techniques

6 · Advanced SQL Techniques — Complete Reference Guide (Banking Project)

Table of Contents

1. Subqueries

→ Task 1: Find accounts with a balance greater than the average balance of all accounts (Use Subquery).

```
SELECT account_id, account_type, account_balance
FROM Accounts
WHERE account_balance > (
    SELECT AVG(account_balance) FROM Accounts
);
```

→ Task 2: Find customers who own at least one account (Use Subquery with IN).

```
SELECT customer_id, first_name, last_name, customer_segment
FROM Customers
WHERE customer_id IN (
    SELECT customer_id FROM Accounts
);
```

→ Task 3: Find the highest balance recorded at each account's branch (Use Correlated Subquery).
Additionally, provide details such as Account ID and Branch Name.

```
SELECT
    a1.account_id, a1.branch_name, a1.account_balance,
    (
        SELECT MAX(a2.account_balance)
        FROM Accounts a2
        WHERE a2.branch_name = a1.branch_name
    ) AS branch_highest_balance
FROM Accounts a1;
```

2. CTE (Common Table Expressions)

→ Task 4: Find customers whose total balance across all accounts is greater than 500000 (Use CTE).

```
WITH CustomerTotals AS (
    SELECT
        c.customer_id, c.first_name, c.last_name,
        SUM(a.account_balance) AS total_balance
    FROM Customers c
    INNER JOIN Accounts a
        ON c.customer_id = a.customer_id
    GROUP BY c.customer_id, c.first_name, c.last_name
)
SELECT *
FROM CustomerTotals
WHERE total_balance > 500000;
```

→ Task 5: Find the top account by balance in each customer segment (Use CTE with Window Function).

```
WITH RankedAccounts AS (
    SELECT
        c.customer_segment, a.account_id, a.account_balance,
        ROW_NUMBER() OVER (
            PARTITION BY c.customer_segment
            ORDER BY a.account_balance DESC
        ) AS rn
    FROM Customers c
    INNER JOIN Accounts a
        ON c.customer_id = a.customer_id
)
SELECT *
FROM RankedAccounts
WHERE rn = 1;
```

→ **Task 6: Calculate the net cash flow (total deposits minus total withdrawals) for each account (Use Multiple CTEs).**

```
WITH Deposits AS (
    SELECT account_id, SUM(transaction_amount) AS total_deposits
    FROM Transactions
    WHERE transaction_type = 'Deposit'
    GROUP BY account_id
),
Withdrawals AS (
    SELECT account_id, SUM(transaction_amount) AS total_withdrawals
    FROM Transactions
    WHERE transaction_type = 'Withdrawal'
    GROUP BY account_id
)
SELECT
    d.account_id, d.total_deposits, w.total_withdrawals,
    d.total_deposits - ISNULL(w.total_withdrawals, 0)
        AS net_cash_flow
FROM Deposits d
LEFT JOIN Withdrawals w
    ON d.account_id = w.account_id;
```

3. Views

→ **Task 7: Create a view showing the total balance for each customer (Create View).**

```
CREATE VIEW vw_CustomerTotalBalance AS
SELECT
    c.customer_id, c.first_name, c.last_name, c.customer_segment,
    SUM(a.account_balance) AS total_balance
FROM Customers c
INNER JOIN Accounts a
    ON c.customer_id = a.customer_id
GROUP BY c.customer_id, c.first_name, c.last_name, c.customer_segment;

-- Using the view is now as easy as:
SELECT * FROM vw_CustomerTotalBalance;
```

→ **Task 8: Create a view showing accounts that have never had a transaction (Create View).**

```
CREATE VIEW vw_DormantAccounts AS
SELECT
    a.account_id, a.account_type, a.account_status, a.open_date
FROM Accounts a
LEFT JOIN Transactions t
    ON a.account_id = t.account_id
WHERE t.transaction_id IS NULL;

-- Using the view is now as easy as:
SELECT * FROM vw_DormantAccounts;
```

→ **Task 9: Create a view showing successful transactions along with customer and account details (Create View).**

```
CREATE VIEW vw_SuccessfulTransactionFeed AS
SELECT
    t.transaction_id, t.transaction_date, t.transaction_amount,
    t.transaction_channel, a.account_type,
    c.first_name, c.last_name, c.customer_segment
FROM Transactions t
INNER JOIN Accounts a
    ON t.account_id = a.account_id
INNER JOIN Customers c
    ON a.customer_id = c.customer_id
WHERE t.transaction_status = 'Success';

-- Now filtering by channel is a one-line query:
SELECT * FROM vw_SuccessfulTransactionFeed
WHERE transaction_channel = 'Mobile App';
```

4. CTAS & Temp Tables

→ **Task 10: Create a new table containing only Premium-segment customers (Use CTAS).**

```
SELECT
    customer_id, first_name, last_name,
    customer_segment, annual_income
INTO PremiumCustomers
FROM Customers
WHERE customer_segment = 'Premium';
```

→ Task 11: Store the total transaction amount for each account in a temporary table (Use Temp Table).
Additionally, find accounts with total transactions greater than 200000, then drop the temp table.

```
SELECT
    account_id, SUM(transaction_amount) AS total_txn_amount
INTO #AccountActivity
FROM Transactions
GROUP BY account_id;

SELECT
    a.account_id, a.account_type, act.total_txn_amount
FROM Accounts a
INNER JOIN #AccountActivity act
    ON a.account_id = act.account_id
WHERE act.total_txn_amount > 200000;

DROP TABLE #AccountActivity;
```

→ Task 12: Create a new permanent table combining each account's details with its total transaction amount (Use CTAS).

```
SELECT
    a.account_id, a.account_type, a.account_balance,
    ISNULL(SUM(t.transaction_amount), 0) AS total_transacted
INTO AccountSummary
FROM Accounts a
LEFT JOIN Transactions t
    ON a.account_id = t.account_id
GROUP BY a.account_id, a.account_type, a.account_balance;
```

5. Stored Procedures

→ Task 13: Create a stored procedure to retrieve customers based on a given customer segment (Create Stored Procedure).

```
CREATE PROCEDURE GetCustomersBySegment
    @Segment VARCHAR(20)
AS
BEGIN
    SELECT customer_id, first_name, last_name, customer_segment
    FROM Customers
    WHERE customer_segment = @Segment;
END;

-- Example of running it:
EXEC GetCustomersBySegment @Segment = 'Premium';
```

→ **Task 14: Create a stored procedure to retrieve the transaction history for a given Account ID (Create Stored Procedure).**

```
CREATE PROCEDURE GetAccountTransactionHistory
    @AccountId INT
AS
BEGIN
    SELECT
        transaction_id, transaction_date, transaction_type,
        transaction_amount, transaction_status
    FROM Transactions
    WHERE account_id = @AccountId
    ORDER BY transaction_date;
END;

-- Example of running it:
EXEC GetAccountTransactionHistory @AccountId = 1001;
```

→ **Task 15: Create a stored procedure to retrieve accounts with a balance below a given amount (Create Stored Procedure).**

```
CREATE PROCEDURE GetAccountsBelowBalance
    @MinBalance DECIMAL(15,2)
AS
BEGIN
    SELECT account_id, account_type, account_balance
    FROM Accounts
    WHERE account_balance < @MinBalance;
END;

-- Example of running it:
EXEC GetAccountsBelowBalance @MinBalance = 10000;
```

6. Triggers

→ Task 16: Create a trigger that automatically logs every new transaction into an audit table (Create Trigger).

```
CREATE TABLE TransactionAuditLog (
    audit_id INT IDENTITY(1,1) PRIMARY KEY,
    transaction_id INT,
    account_id INT,
    transaction_amount DECIMAL(12,2),
    logged_at DATETIME DEFAULT GETDATE()
);

CREATE TRIGGER trg_LogNewTransaction
ON Transactions
AFTER INSERT
AS
BEGIN
    INSERT INTO TransactionAuditLog
        (transaction_id, account_id, transaction_amount)
    SELECT transaction_id, account_id, transaction_amount
    FROM inserted;
END;
```

→ Task 17: Create a trigger that prevents inserting a transaction with a negative amount (Create Trigger).

```
CREATE TRIGGER trg_PreventNegativeAmount
ON Transactions
AFTER INSERT
AS
BEGIN
    IF EXISTS (
        SELECT 1 FROM inserted WHERE transaction_amount < 0
    )
    BEGIN
        RAISERROR('Transaction amount cannot be negative.', 16, 1);
        ROLLBACK TRANSACTION;
    END
END;
```

→ Task 18: Create a trigger that automatically updates the account balance when a new transaction is inserted (Create Trigger).

```
CREATE TRIGGER trg_UpdateAccountBalance
ON Transactions
AFTER INSERT
AS
BEGIN
    UPDATE a
    SET a.account_balance = a.account_balance + i.transaction_amount
    FROM Accounts a
    INNER JOIN inserted i
        ON a.account_id = i.account_id
    WHERE i.transaction_type IN ('Deposit', 'Interest Credit');
END;
```